Syrian Private University

Faculty of Artificial Intelligence Engineering

Department of Systems Security and Computer Networks

Secure Chat-Based Environment Using AI-Based IDS

Prepared by:

William Rateb Albeni

Supervised by:

Dr. Wassim Aljuneidi

Year

2025-2026

Table of Contents

Table of Figures

List of Abbreviations and Scientific Terms

AES	Advanced Encryption Standard
API	Application Programming Interface
BLL	Business Logic Layer
DAL	Data Access Layer
DB	Database
DBA	Database Administrator
DTO	Data Transfer Object
E2EE	End-to-End Encryption
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IRC	Internet Relay Chat
JSON	JavaScript Object Notation
JWT	JSON Web Token
MITM	Man-in-the-Middle
PKCS	Public-Key Cryptography Standards
RSA	Rivest-Shamir-Adleman
SQL	Structured Query Language
SSL	Secure Sockets Layer
SSMS	SQL Server Management Studio
TLS	Transport Layer Security

Abstract

The rapid evolution of digital communication necessitates robust security mechanisms to protect sensitive user data. This project aims to design and implement a secure, real-time chat application utilizing a modern N-Tier architecture. To achieve this objective, a full-stack system was developed, comprising a React-based frontend and a C# .NET Core backend. Real-time bidirectional communication and dynamic message status updates (sent, delivered, read) were achieved using SignalR technology. Furthermore, to ensure data confidentiality against unauthorized database access, a Data-at-Rest encryption model was implemented using the Advanced Encryption Standard (AES) with a server-managed master key. Experimental testing demonstrated the system's high efficiency in delivering real-time messages with minimal latency, while successfully protecting stored data in the SQL Server database. The proposed architecture provides a practical balance between stringent security requirements and functional usability, offering a reliable platform for secure enterprise communication.

Keywords: Secure Chat, Real-Time Communication, SignalR, Data-at-Rest Encryption, N-Tier Architecture.

يحتم التطور السريع للاتصالات الرقمية وجود آليات أمنية قوية لحماية بيانات المستخدمين الحساسة. يهدف هذا المشروع إلى تصميم وتنفيذ تطبيق محادثة آمن في الوقت الفعلي باستخدام معمارية (N-Tier) حديثة. ولتحقيق هذا الهدف، تم تطوير نظام متكامل يتكون من واجهة أمامية (Frontend) مبنية باستخدام (React) وبرمجة (Backend) باستخدام (C# .NET Core) . تم تحقيق الاتصال ثنائي الاتجاه في الوقت الفعلي وتحديثات حالة الرسائل الديناميكية (مرسلة، تم التسليم، مقروءة) باستخدام تقنية (SignalR) . علاوة على ذلك، ولضمان سرية البيانات ضد الوصول غير المصرح به إلى قاعدة البيانات، تم تنفيذ نموذج تشفير البيانات في حالة السكون (Data-at-Rest) باستخدام معيار التشفير المتقدم (AES) مع مفتاح رئيسي يدار من قِبل الخادم. أثبتت الاختبارات التجريبية الكفاءة العالية للنظام في تسليم الرسائل في الوقت الفعلي بأقل زمن انتقال، مع حماية البيانات المخزنة بنجاح في قاعدة بيانات .(SQL Server) توفر المعمارية المقترحة توازناً عملياً بين المتطلبات الأمنية الصارمة وقابلية الاستخدام الوظيفي، مما يقدم منصة موثوقة للاتصالات المؤسسية الآمنة.

الكلمات المفتاحية: محادثة آمنة، الاتصال في الوقت الفعلي، تقنية SignalR، تشفير البيانات في حالة السكون، المعمارية متعددة الطبقات

Chapter One
Introduction

1-1. Preface

The rapid advancement of digital communication technologies has fundamentally transformed how individuals and organizations exchange information. In the modern era, real-time messaging platforms have become the backbone of daily operations, necessitating highly responsive and available network infrastructures. However, this heavy reliance on digital communication introduces critical vulnerabilities, making information security a primary concern. The increase of cyber threats, ranging from data interception in transit to advanced database breaches, demands the implementation of robust security architectures. Consequently, modern software engineering must bridge the gap between seamless real-time communication and strict data protection mechanisms. This project explores the development of a secure chat application that integrates modern web technologies, specifically a React-based frontend and an ASP.NET Core backend, with real-time bidirectional communication protocols and robust cryptographic standards.

1-2. Scientific Problem and Project Justifications

A fundamental challenge in modern network security is balancing absolute data privacy with functional usability. Traditional communication systems often store user messages in plaintext within databases, rendering them highly susceptible to mass data exfiltration if the database server is exposed. Conversely, implementing strict End-to-End Encryption (E2EE) prevents the server from accessing the data entirely, which complicates essential server-side functionalities such as message search capabilities, and centralized auditing required in enterprise environments. The scientific problem addressed in this project is the vulnerability of stored communication data to unauthorized database access. The justification for this project lies in the necessity to protect sensitive information from database leaks while maintaining server-side functional flexibility. To resolve this, the project adopts a Data-at-Rest encryption architecture using the Advanced Encryption Standard (AES) with server-managed keys. This approach ensures that if the database is breached, the exfiltrated data remains encrypted and unreadable, providing a crucial layer of security known as Defense in Depth.

1-3. Project Objectives

The primary objective of this project is to design and implement a secure, real-time communication platform based on a clean N-Tier architecture. This overarching goal is achieved through the following specific objectives:

- Developing a responsive frontend using React and a robust backend utilizing C# .NET Core.
- Enforcing a strict N-Tier architecture (Presentation, Business Logic, and Data Access layers) to ensure code maintainability, separation of concerns, and system scalability.
- Integrating SignalR technology to facilitate real-time, bidirectional communication, ensuring minimal latency in message delivery.
- Securing sensitive data by applying Data-at-Rest encryption using AES, preventing unauthorized access to the underlying SQL Server database.
- Securing data in transit by enforcing HTTPS/TLS protocols, mitigating the risk of Man-in-the-Middle (MITM) attacks.

1-4. Previous Studies and State-of-the-Art

Historically, early internet relay chat (IRC) protocols and fundamental messaging applications operated without built-in encryption, leaving data exposed both in transit and at rest. Subsequent studies and industrial advancements led to the widespread adoption of the Transport Layer Security (TLS) protocol, which successfully mitigates interception during data transmission. Recently, the industry has diverged into two primary architectural models for data protection. The first model is End-to-End Encryption (E2EE), utilized by applications like WhatsApp and Signal, which provides zero-knowledge privacy but limits server utility. The second model, which this project builds upon, is the Enterprise Encryption Architecture (Encryption at Rest), utilized by platforms such as Slack, Microsoft Teams, and cloud-based Telegram. State-of-the-art implementations in this domain utilize symmetric encryption algorithms, predominantly AES-256, combined with secure key management practices. By separating the cryptographic keys from the physical database storage, these modern systems achieve a high level of security against database theft while preserving the server's ability to process and synchronize data efficiently across multiple user devices.

1-5. Challenges

Developing a comprehensive secure chat system presents several technical and architectural challenges. A primary challenge is the seamless integration of real-time protocols (SignalR) with a stateless HTTP architecture. Managing client connection states and ensuring accurate, instant synchronization of message delivery statuses across disconnected or intermittently connected clients requires complex event handling and state management. Furthermore, strictly maintaining the N-Tier architectural pattern gives a challenge, as it necessitates the careful transfer of Data Transfer Objects (DTOs) between layers without exposing the underlying database context or

entity models to the presentation layer. Additionally, securely managing the database master key and ensuring cryptographic operations do not introduce significant latency into the real-time communication flow required correct optimization.

1-6. Practical Applications

The architecture and methodologies developed in this project have significant practical applications, particularly within enterprise and organizational environments. The implementation of server-managed Data-at-Rest encryption makes this system highly suitable for internal corporate communications, where administrators require the ability to audit and synchronize data, but must simultaneously protect the organization from external database breaches. Furthermore, the model is applicable to sectors requiring strict regulatory compliance regarding data storage, such as healthcare communication portals and financial advisory services, where data integrity, real-time responsiveness, and defense against database theft are critical operational requirements.

1-7. Chapter Summary

This chapter provided a comprehensive introduction to the project, outlining the context of modern secure communications. The scientific problem regarding database vulnerabilities was identified, justifying the adoption of a Data-at-Rest encryption model. The specific objectives of implementing an N-Tier, real-time, and encrypted application were detailed. Additionally, the chapter reviewed the state-of-the-art approaches in the field, highlighted the primary technical challenges encountered during development, and discussed the practical applications of the proposed system.

Chapter Two
Theoretical Framework

2-1. Preface

This chapter builds the theoretical foundations and academic concepts necessary for developing a secure communication system. It explores the underlying principles of cryptographic protocols, real-time system architectures, and the nature of cyber threats such as Man-in-the-Middle attacks.

2-2. Cryptographic Protocols and Data Protection

Cryptography is the fundamental science of securing information and communications through mathematical codes, ensuring that only intended recipients can access and process the data. Modern secure messaging platforms use advanced encryption protocols to protect transmitted data from interception by malicious actors. Theoretical approaches to data security differentiate between End-to-End Encryption (E2EE) and Enterprise Encryption architectures. In Enterprise settings, Data-at-Rest encryption relies on server-managed keys to encrypt stored data, providing a practical balance between security against database breaches and essential operational usability. A widely adopted standard for this purpose is the Advanced Encryption Standard (AES), a symmetric block cipher utilized to securely encrypt sensitive message payloads before they are stored in the database.[2]

2-3. Data Integrity and Authentication Mechanisms

Data integrity refers to the assurance that information remains accurate, consistent, and unaltered during both storage and transmission. Detecting unauthorized modifications requires algorithms capable of evaluating structural data integrity in real-time. Beyond securing messages in transit, ensuring data integrity at the database level is important for secure user authentication. This is achieved through cryptographic hashing functions, such as Bcrypt, which transform plaintext user passwords into irreversible hash values. By applying a unique cryptographic salt to each password before hashing, the system effectively prevent brute-force and dictionary attacks, ensuring that even if database records are compromised, the actual user credentials remain hard to retrieve and mathematically protected.[4]

2-4. Threat Models and Adversarial Attacks

A critical vulnerability in network communications is the Man-in-the-Middle (MITM) attack, where malicious entities intercept and modify the data flowing between communicating users. Defending against these intrusions requires platforms capable of safeguarding conversations from unauthorized access and interception. In modern contexts involving automated systems, MITM threats have evolved into adversarial attacks that intercept and manipulate user messages before they reach backend processors. These adversarial frameworks operate by injecting false

information, misleading instructions, or unrelated noise into the communication process. Such attacks exploit the input sensitivity of systems, weakening the actual correctness of generated responses and highlighting the necessity for monitoring response uncertainty to detect potential compromises.[5]

2-5. Real-Time Communication Architectures

The development of highly responsive secure messaging solutions relies on modular software architectures. Client-side interfaces are typically constructed using modern web frameworks to provide a seamless user experience, while backend processing manages data, productivity, and validation. To enable real-time bidirectional communication, these platforms utilize socket-based technologies, such as WebSockets or Socket.io, which simplify instant message transmission and dynamic status updates. Cloud-based backends, such as Firebase, offer scalable secure user authentication, real-time database synchronization, and encrypted storage capabilities.[1]

2-6. Chapter Summary

This chapter provided an overall review of the theoretical models and algorithms that constitute modern secure communication frameworks. It detailed the mechanics of MITM and adversarial attacks, underscoring the vulnerabilities present in real-time data transmission, and the theoretical foundations of N-Tier architectures.

Chapter Three
Environment Setup and Implementation

3-1. Preface

This chapter provides a detailed description of the practical implementation of the secure chat project. It outlines the development environment, software components, and the specific configurations applied to build the system. A major focus is placed on the practical execution of the security mechanisms, detailing how data is protected at rest and in transit, how user identities are secured, and how system availability is maintained. The configurations are designed to meet high security standards while keeping the application functional and efficient.

3-2. Development Environment and Tools

To implement the proposed architecture, a modern and robust set of development tools and frameworks was utilized. The backend services and application logic were developed using C# and the ASP.NET Core framework, providing a high-performance environment suitable for handling real-time operations. The database management system selected was Microsoft SQL Server, used to store user profiles and encrypted messages. For the frontend, React.js was chosen with Vite as the build tool, ensuring a fast, responsive, and secure user interface.

3-3. Data Security at Rest (Encryption at Rest)

A primary security requirement for the chat application is to protect sensitive messages stored in the database from unauthorized access, especially in the event of a database leak or physical server compromise. To achieve this, a Data-at-Rest encryption strategy was implemented using the Advanced Encryption Standard (AES), a widely trusted symmetric encryption algorithm.

In this implementation, the backend server acts as the central authority for encrypting and decrypting data before it reaches the SQL Server database. A highly secure key, named the DbMasterKey, was generated and configured within the server's environment settings (appsettings.json).

When a message is received by the server, the application logic uses the DbMasterKey to encrypt the message content using AES before executing the database INSERT command. Consequently, the database only stores the ciphertext. When a user requests to view their messages, the server retrieves the ciphertext, decrypts it using the same DbMasterKey, and sends the readable text back to the authorized user. This separation of the storage medium (SQL Server) and the encryption key (managed by the ASP.NET Core server) adds a strong layer of defense. If an attacker successfully

steals the database files, they cannot read the messages without also compromising the backend server to obtain the DbMasterKey.

This figure shows the AES encryption process before saving to the database:

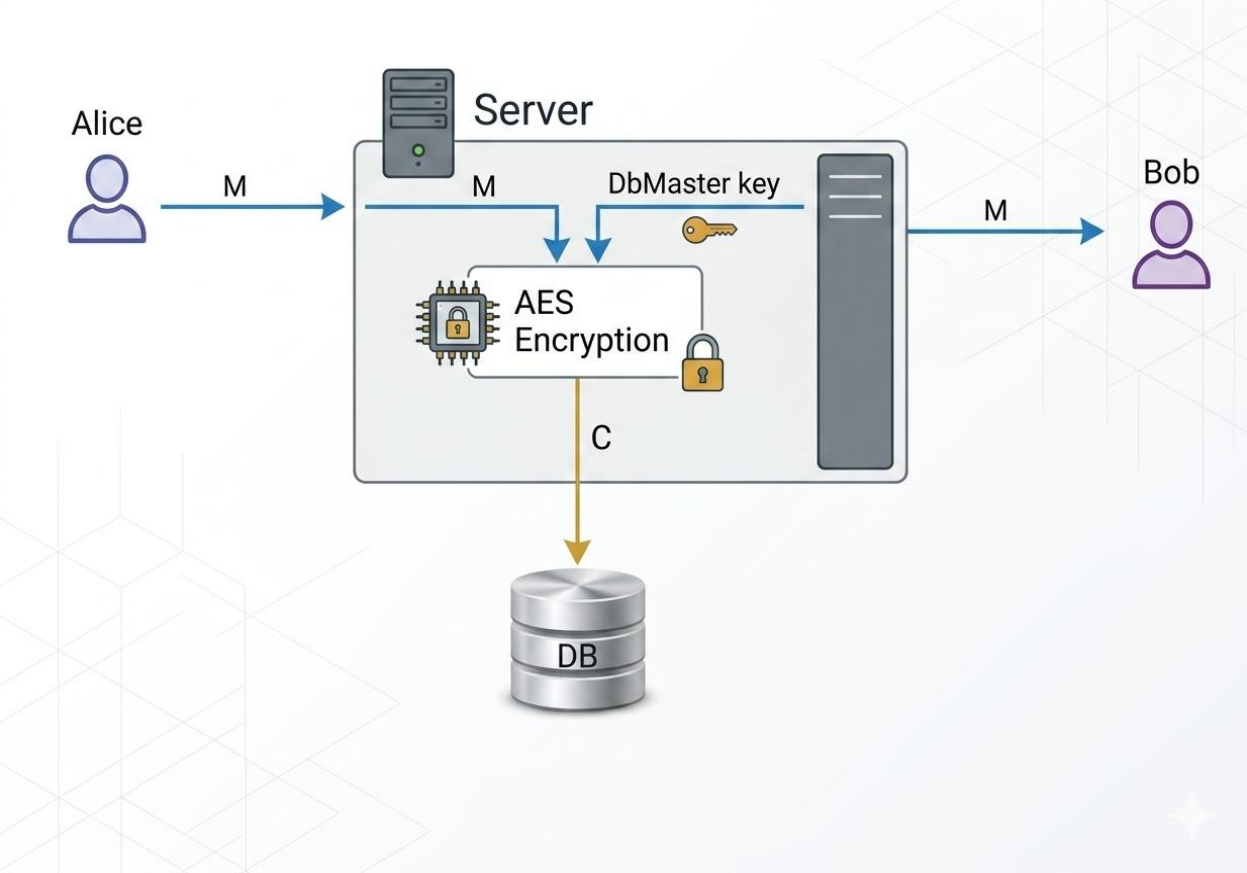

Figure 3-1: AES Encryption Process

3-4. Password Security and Integrity (Hashing)

Protecting user credentials is as critical as protecting the messages. Passwords must never be stored in plain text, nor should they be encrypted, as encryption can be reversed. Instead, a strong cryptographic hashing mechanism was implemented to ensure password integrity and security.

The chosen hashing algorithm for this project was **Bcrypt**. Bcrypt is specifically designed for password hashing because it combines an adaptive cost factor, making the hashing process intentionally slow. This slowness is a critical defense mechanism against brute-force and dictionary attacks, as it makes guessing passwords computationally expensive for attackers. Additionally, Bcrypt automatically generates a unique random "salt" for every password. This salt ensures that even if two users have the exact same password, their resulting hashes will look completely different in the database, protecting against pre-computed hash attacks (Rainbow Tables).

To increase the security even further, an additional layer known as a **Pepper** was implemented. A pepper is a long, secret cryptographic key that is securely stored in the backend server, completely separate from the database. Before a user's password is hashed with Bcrypt, the pepper is appended to the password.

The practical execution flow operates as follows: First, the user inputs their password. Second, the backend server combines the password with the secret pepper. Third, the combined string is hashed using Bcrypt with its unique salt. Finally, only the resulting hash is stored in the SQL Server database. This implementation guarantees that if a hacker tries to steal the database, they still cannot crack the passwords because they do not have the secret pepper, which remains safely isolated in the application server.

Validating the password for user is done as follows: First, when logging in the system, the user password is inserted, BCrypt algorithm takes the password and add the pepper to it. Second, the hash process is applied on the combination, then the user’s salt is being retrieved from the database and added it to the front of the previous hash. Finally, the resulting hash is compared with the hash in the database giving a corresponding action.

This figure shows the password storing process and BCrypt workflow:

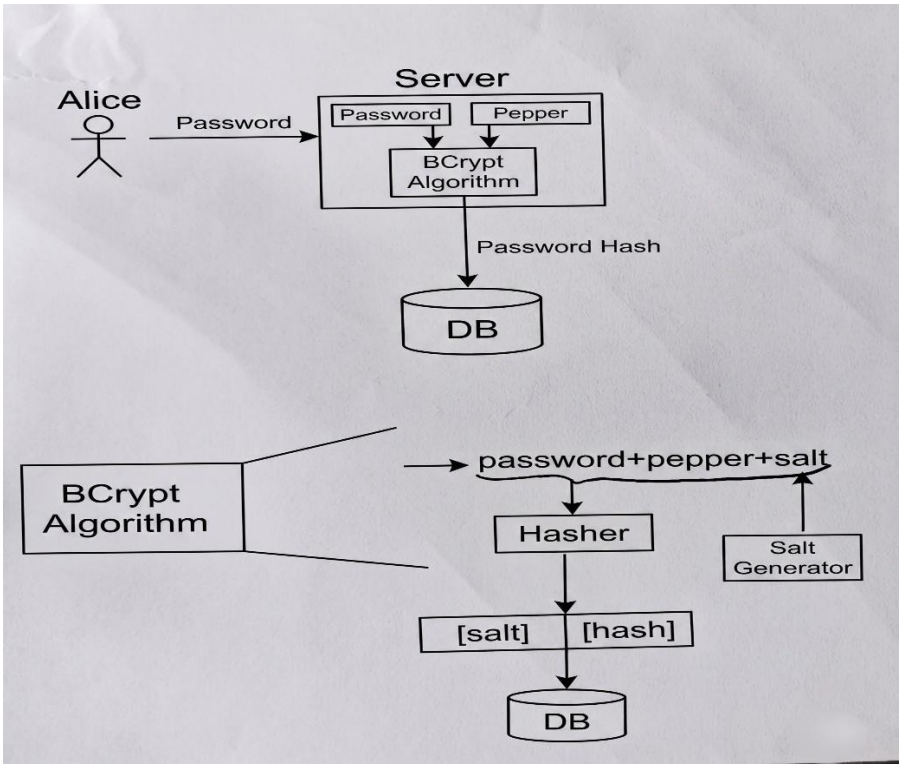

Figure 3-2: BCrypt Algorithm Process

And the following figure shows the identification and password validating process:

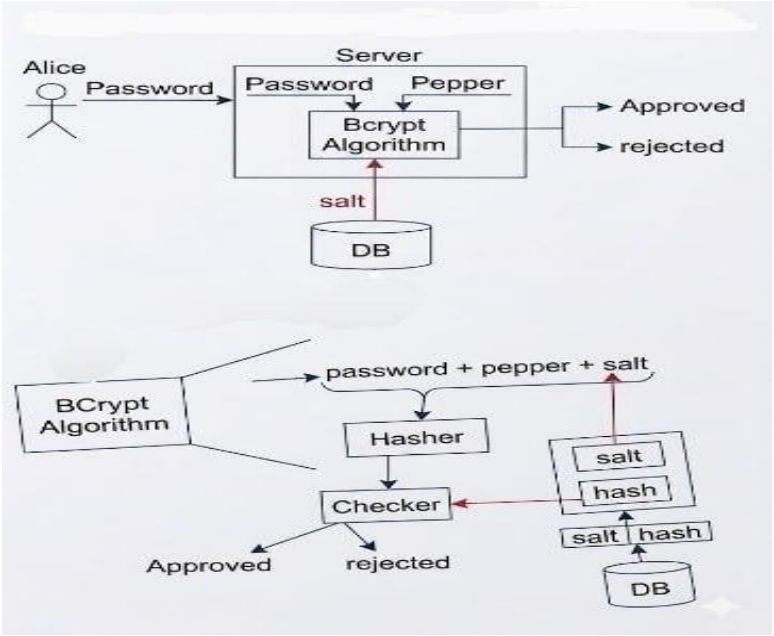

Figure 3-3: Identification/ Password Validating

3-5. Data Security in Transit (Encryption in Transit)

While securing data at rest is crucial, protecting data as it travels across the network is equally important to prevent eavesdropping and Man-in-the-Middle (MITM) attacks. To achieve this, strict Encryption in Transit policies were enforced across the entire application.

Hypertext Transfer Protocol Secure (HTTPS) was configured and mandated for both the React frontend and the ASP.NET Core backend. At the network level, HTTPS secures the communication over Layer 4 (the Transport Layer) by utilizing the Transport Layer Security (TLS) protocol. By enforcing HTTPS, all HTTP headers, request bodies, and server responses are fully encrypted before leaving the device. The backend was explicitly configured to reject any unencrypted HTTP requests, automatically redirecting them to the secure HTTPS port. This implementation ensures that even if an attacker successfully intercepts the network traffic, they will only capture unreadable, encrypted data packets.

This figure shows the HTTPS traffic form:

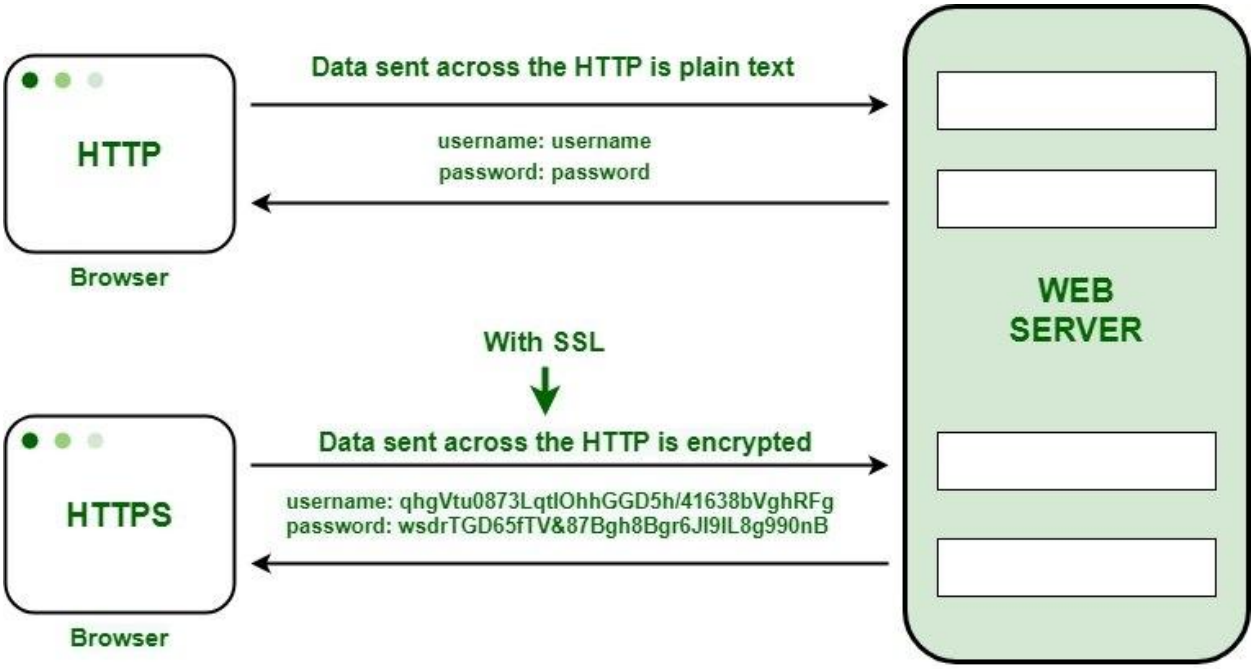

Figure 3-4: HTTPS traffic

3-6. Advanced Data Protection (Hybrid Encryption: AES + RSA)

To provide an additional layer of security beyond the transport layer, an application-layer Hybrid Encryption model was implemented. This model combines the performance efficiency of symmetric encryption with the secure key distribution capabilities of asymmetric encryption, establishing a highly secure channel for message data.

The mechanism utilizes both the Advanced Encryption Standard (AES) and the Rivest-Shamir-Adleman (RSA) algorithms. AES is exceptionally fast and is optimally suited for encrypting the actual content of the chat messages. However, sharing the AES key securely over a network gives an important challenge. To resolve this, RSA is utilized. The process is executed as follows:

First, a dynamic, random AES session key is generated for the message payload. The message is then encrypted using this AES key. Next, the AES key itself is encrypted using the recipient's public RSA key. Both the encrypted message payload and the encrypted AES key are transmitted to the server. Upon reception, the recipient utilizes their private RSA key to decrypt the AES key, which is used to decrypt the original message. This hybrid approach guarantees maximum security for data payloads, ensuring that only the intended recipient can read the message, without compromising real-time communication speed.

This figure demonstrates the hybrid encryption process:

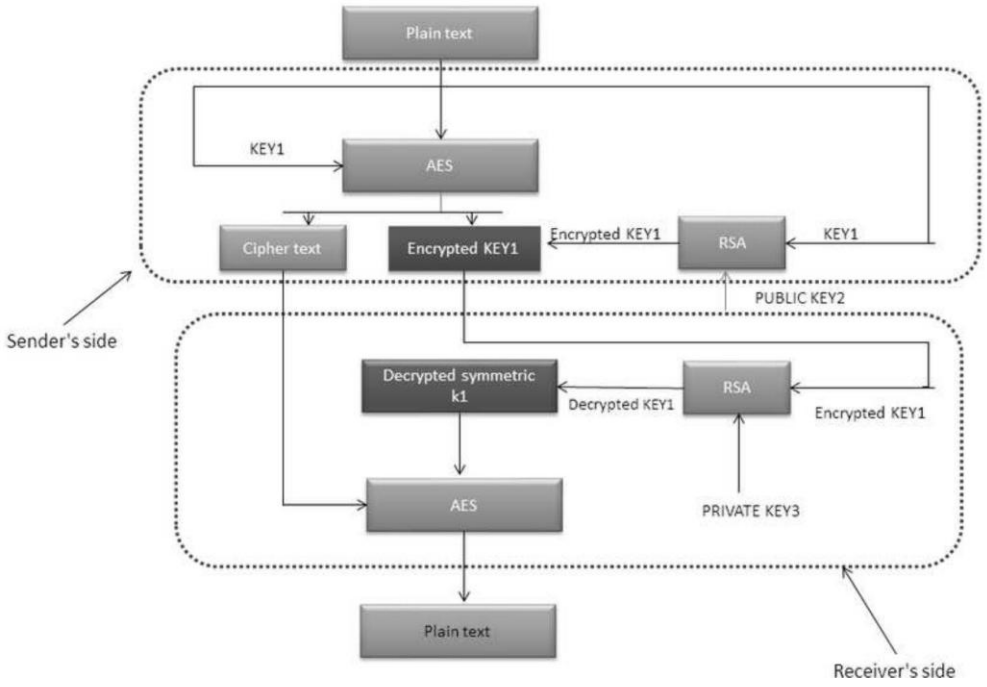

Figure 3-5: Hybrid Encryption

3-7. Achieving System Availability

In the domain of information security, Availability is a core element alongside Confidentiality and Integrity. It ensures that the secure chat application remains operational, reliable, and accessible to authorized users whenever needed. To achieve high availability, several practical configurations were integrated into the system.

For the real-time communication managed by SignalR, automatic reconnection logic was implemented on the client side. In the event of a temporary network drop or instability, the frontend application automatically attempts to restore the WebSocket connection smoothly, without requiring manual user involvement or application restarts. On the backend, the adoption of a clean N-Tier architecture ensures that system components are loosely coupled. Consequently, a failure in one specific module (e.g., a specific business logic service) is less likely to cause a complete server crash. Additionally, database connection was configured within the Entity Framework to automatically retry failed database operations caused by the passing network glitches, minimizing system downtime and ensuring a continuous user experience.

3-8. Chapter Summary

This chapter detailed the practical setup and the security implementations of the project. It outlined the configuration of the development environment and highlighted the methodologies utilized to secure data at rest via AES and the DbMasterKey. Furthermore, the techniques for password hashing using Bcrypt combined with a secret pepper were explained to demonstrate database integrity. The chapter also discussed the enforcement of HTTPS for securing data in transit across Layer 4, and the implementation of a robust hybrid AES-RSA encryption model for payload protection. Finally, the practical steps taken to guarantee system availability and real-time connection flexibility were presented.

Chapter Four
Proposed Solution

4-1. Preface

This chapter presents a detailed explanation of the proposed security architecture for the secure chat application. The primary focus is placed on the security methodologies, cryptographic mechanisms, and the logical flow of protected data across the network. The chapter explains how the system is designed to mitigate cyber threats, enforce data confidentiality, and maintain secure real-time communication. The functional security mechanisms are supported by architectural diagrams and flowcharts to illustrate the complex defense layers implemented within the system.

4-2. Secure System Architecture (N-Tier Defense-in-Depth Model)

To minimize the external attack surface and enforce a strict defense-in-depth strategy, the proposed solution is constructed utilizing a secure N-Tier architecture. Rather than focusing just on software organization, this architectural choice is used primarily as a security boundary to physically and logically isolate the database from direct internet exposure.

The security architecture is divided into three isolated zones:

- **The Presentation Zone (External Facing):** This layer acts as the client-side interface and the primary entry point for user interaction. It is strictly designated as an untrusted zone. No cryptographic master keys or sensitive database connection strings are stored here. Its only security responsibility is to securely capture user inputs and establish an encrypted tunnel with the server.

- **The Cryptographic Boundary (Business Logic Layer):** Operating on the backend server, this layer serves as the core trust zone of the system. All critical security operations, including payload inspection, token validation, AES encryption/decryption processes, and Bcrypt password hashing, are executed exclusively within this boundary. By centralizing the security logic here, malicious client-side manipulations are effectively neutralized.

- **The Isolated Data Zone (Data Access Layer):** The database server is placed in a highly restricted internal network zone. It does not have any public-facing endpoints. Communication with this layer is exclusively permitted from the verified Cryptographic Boundary. This strict access control mechanism ensures that direct SQL injection attempts or unauthorized data extraction attacks are strongly mitigated.

This figure shows the security of N-Tier Architecture:

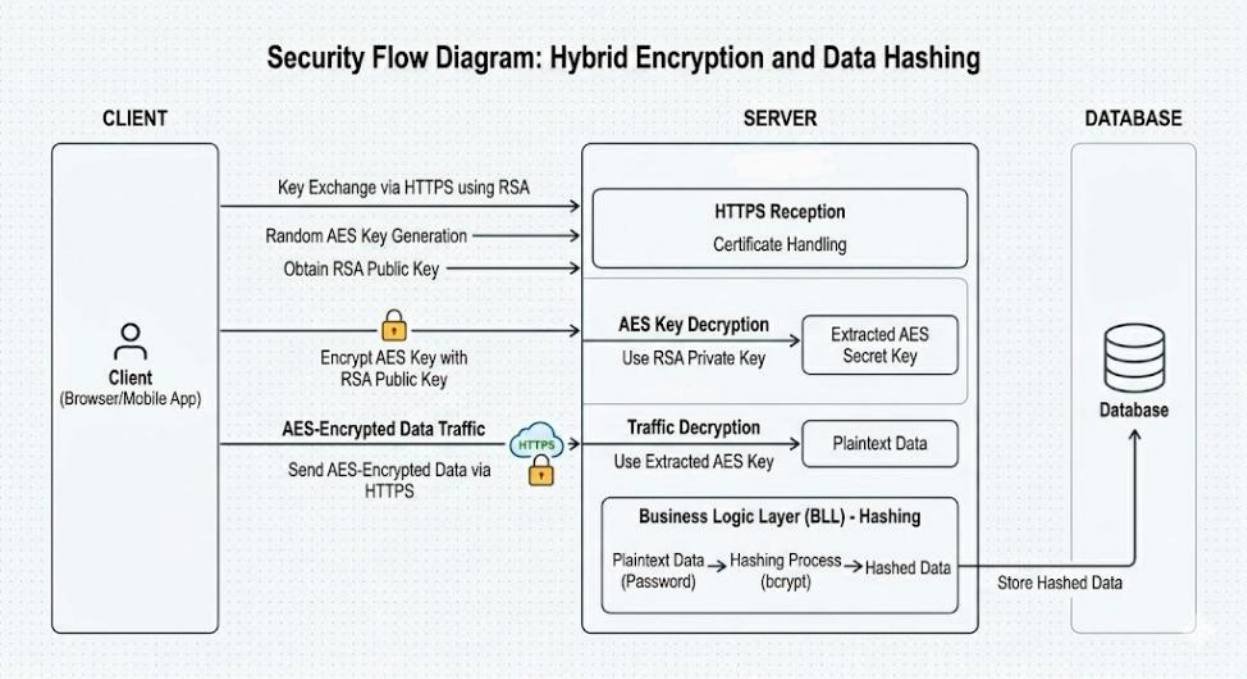

Figure 4-1: Security of N-Tier Architecture System

4-3. Secure Real-Time Communication Workflow

Providing instant messaging requires maintaining persistent, open connections between the clients and the server. From a cybersecurity perspective, maintaining persistent connections introduces risks such as session hijacking and unauthorized payload injection. To mitigate these risks securely, a highly controlled real-time routing mechanism was proposed.

The workflow is designed to ensure that data confidentiality and access control are continuously enforced during the real-time exchange. The process operates through the following secure sequence:

1. **Secure Session Establishment:** When a user authenticates, a persistent WebSocket connection is established over a TLS-encrypted tunnel. The server maps the authenticated user's identity to a unique, randomized connection ID. This prevents attackers from guessing connection identifiers to hijack sessions.

2. **Encrypted Payload Transmission:** When a message is sent, the payload arrives at the server already wrapped in transport-layer encryption. The server acts as a secure router, it does not broadcast the message blindly.

3. **Access Control Verification:** The server verifies the sender's authorization token and checks the intended recipient's status. The message is then subjected to Data-at-Rest encryption (using AES) before being routed to the database.

4. **Targeted Delivery:** If the recipient maintains an active, authenticated session, the encrypted payload is pushed directly and exclusively to their specific connection ID. If the recipient is offline, the message is securely held in the isolated database until a verified session is re-established.

This routing methodology ensures that messages are delivered instantly while strictly maintain access control policies, ensuring that no unauthorized client can listen to or intercept the real-time data stream.

4-4. Hybrid Encryption Security Logic (AES and RSA Integration)

To ensure the absolute confidentiality of message payloads and to resolve the vulnerability of secure key distribution over a network, a hybrid encryption protocol was designed. Hybrid encryption schemes that combine symmetric algorithms like AES with asymmetric algorithms such as RSA are utilized for optimal performance and security. This methodology guarantees that the message content remains secure from interception, even if the primary server infrastructure is compromised.

The security logic behind this integration is based on using the strengths of both algorithms. AES is utilized for encrypting the actual message content due to its high computational speed and efficiency with large data blocks. However, sharing the symmetric AES key across the network securely presents a critical risk. To mitigate this, RSA asymmetric encryption is used exclusively for key exchange.

The secure transmission process is executed through the following strict steps:

1. **Key Generation:** A unique, randomized AES session key is generated locally by the sender's client for each individual message.
2. **Payload Encryption:** The plain text message is encrypted using this newly generated AES key.
3. **Key Encapsulation:** The symmetric AES key is then cryptographically encapsulated by encrypting it with the recipient's public RSA key.

4. **Secure Transmission:** The combined payload (the AES-encrypted message and the RSA-encrypted AES key) is transmitted to the server.
5. **Decryption:** Upon delivery, the recipient utilizes their strictly guarded private RSA key to decrypt the AES key. Then, the decrypted AES key is used to decrypt the original message.

Through this architecture, the hybrid encryption supports a secure channel for keys exchange and a small size ciphertexts because of symmetric encryption usage.

4-5. Secure Message State Tracking

The accurate tracking of message states (Sent, Delivered, Read) is a critical operational requirement in modern secure messaging platforms. From a cybersecurity perspective, these state transitions must be strictly protected against manipulation, spoofing, or replay attacks. If a client application were fully trusted to order message states, an attacker could easily fake "Read" receipts for messages they never received or hide "Delivered" notifications to disrupt communication auditing.

To neutralize these vulnerabilities, the proposed solution implements a server-authoritative state tracking mechanism. In this model, the backend Business Logic Layer acts as the absolute judge of all state transitions, and client-side claims are never trusted without cryptographic validation.

The secure state lifecycle is managed as follows:

- **Sent State:** This state is strictly assigned by the server only after the encrypted message payload is successfully validated, processed, and safely stored in the isolated database layer.
- **Delivered State:** This transition is only authorized when the server receives an automated, cryptographically signed acknowledgment directly from the recipient's active WebSocket connection, confirming that the payload has reached the device.
- **Read State:** Transitioning to the "Read" state requires an explicit, authenticated request from the client. This request must be bound to the recipient's secure session token and pass strict access control checks on the server to verify that the user is authorized to update the status of that specific message.

This strict validation process guarantees that message states cannot be artificially manipulated by malicious actors, and with that ensuring the absolute integrity and reliability of the communication timeline.

4-6. Chapter Summary

This chapter provided a comprehensive overview of the proposed solution, detailing the structural and logical methodologies utilized to build a secure, real-time communication platform. The implementation of a secure N-Tier defense-in-depth architecture was explained, highlighting the critical isolation between the external presentation zone and the internal database layer. The secure real-time message routing workflow was outlined, demonstrating how persistent connections are managed safely without exposing the network to unauthorized payload injections. Furthermore, the hybrid encryption logic, integrating AES for high-speed payload encryption and RSA for secure key exchange, was thoroughly detailed. The chapter also presented the server-authoritative message state tracking mechanism, ensuring that message statuses cannot be spoofed by malicious clients.

Chapter Five
Testing, Results, and Comparisons

5-1. Preface

In this chapter, the developed system is evaluated through a series of functional and security tests. The primary objective is to verify the efficiency of the hybrid encryption algorithms and the stability of the real-time communication provided by SignalR. Furthermore, the N-Tier architecture is analyzed to ensure correct data isolation. This chapter concludes with a comparative analysis between the proposed system and previous studies to highlight the development and added value of this research.

5-2. Testing Environment

To ensure accurate and reliable results, the testing process was done in a controlled environment that simulates real-world usage. The environment was divided into two main sections:

5-2.1. Hardware and Software Specifications

The following specifications were utilized during the testing phase:

- **Server-Side:** The backend API was hosted on a local server environment with HTTPS and SSL certificates enabled.
- **Client-Side:** Multiple web browsers were used. The "Incognito" mode was activated to simulate different users on separate devices without session interference.
- **Database:** SQL Server was used, and SQL Server Management Studio (SSMS) was used to monitor the encrypted data directly.

5-2.2. Development Tools

- **Frontend:** React.js (Vite) was used for the user interface.
- **Backend:** .NET (C#) was used for the business logic and SignalR Hub.
- **Encryption Libraries:** Cryptographic libraries were used in both client-side and server-side.

5-3. Testing Methodology

A multi-layered testing strategy was implemented to ensure that the system is free from vulnerabilities and functional errors.

5-3.1. Unit Testing

Every individual module was isolated and tested to ensure it performs its specific task correctly.

- **Encryption Service:** The CryptoService was tested to confirm that any text encrypted with AES can only be recovered using the correct key.
- **Key Exchange:** The RSA key exchange process was verified. It was ensured that the AES session key, once encrypted with the recipient's Public Key (using PKCS1 padding), is successfully decrypted by the recipient's Private Key without any loss of data integrity.
- **Password Hashing:** The registration module was tested to ensure that passwords combined with a Pepper and Salt are correctly hashed using BCrypt. It was verified that identical passwords result in different hashes to prevent rainbow table attacks.

5-3.2. Integration Testing

The interaction between the different layers of the **N-Tier architecture** was verified.

- **SignalR Connectivity:** The connection between the Client and the Hub was tested. It was observed that the server correctly identifies users through their tokens.
- **Data Flow:** The process of sending a message from the Data Access Layer (DAL) to the Business Logic Layer (BLL) and finally to the API was monitored. It was confirmed that no sensitive information (such as plain text passwords or private keys) is leaked during this transmission.

5-3.3. Security Testing

Security testing was the most critical phase of the rating. Several scenarios were simulated to verify the robustness of the protection mechanisms.

- **Eavesdropping Simulation:** Network traffic between the client and the server was intercepted using browser developer tools and network proxies. It was observed that all transmitted data, including chat messages and session keys, appeared as encrypted, unreadable strings. Even if the traffic is captured, the content remains inaccessible without the private keys stored on the users' local devices.

- **Database Inspection (Encryption at Rest):** A direct inspection of the SQL Server database was performed using **SSMS**. The Messages table was examined to check the state of stored information. It was confirmed that the OriginalPayload field does not contain plain text. Instead, it contains data that has been encrypted by the Server's Master Key. This ensures that even a database administrator (DBA) cannot read the private conversations.

- **Unauthorized Access Prevention:** Attempts were made to access the ChatRoom component without a valid JWT Token. The system was observed to redirect unauthorized requests to the login page immediately. Furthermore, the SignalR Hub was tested to ensure it rejects any connection that does not provide a valid identity claim.

This figure shows how the OriginalPayload field stores AES encrypted messages using server's DbMasterKey:

	Id	SenderId	ReceiverId	OriginalPayload
1	B1A2FD8A-6364-49BA-88BD-175BA2228A8A	81CF892A-6053-4D17-B93E-E7DAE0E94593	14654977-03FB-452D-AFC2-6D9F0D9ACE36	3aWZHc4W4MgDfg6/a4qvFlucZUa/MCF7FUV37MsR9dY=
2	A97BB0DD-209C-443B-B80C-1AB2C208B5E7	81CF892A-6053-4D17-B93E-E7DAE0E94593	28207E7B-F177-4CA1-8707-D7A4729E699E	cJm74ILj640S2QqwWYR4vaC9Jj5sZBRliXilhBI9UYg=
3	25A47CA5-19AD-40C8-A9BE-2C56838FC202	14654977-03FB-452D-AFC2-6D9F0D9ACE36	28207E7B-F177-4CA1-8707-D7A4729E699E	ukOhiL3SVArm2i+/lviD1dYDgl8Q+7WBYDdS27FtuQY=
4	B7BD1E42-EBAB-4F89-83E3-41FCE9F77B6A	28207E7B-F177-4CA1-8707-D7A4729E699E	14654977-03FB-452D-AFC2-6D9F0D9ACE36	UPAHGCaO/PBxdf7DOkUHyOiuLiT1FyxHb1Kg0K/R7tw=
5	72EAC629-AEB6-4865-8BFB-614376F8F840	14654977-03FB-452D-AFC2-6D9F0D9ACE36	81CF892A-6053-4D17-B93E-E7DAE0E94593	gVdp7lRKZsSqWCLSDYlJFtdSShwN6yWfHSgvDcn9SE0=
6	FD005549-86E1-4674-B5B9-6A271D150451	28207E7B-F177-4CA1-8707-D7A4729E699E	81CF892A-6053-4D17-B93E-E7DAE0E94593	xrWLD2GtkwQhQYszSZn/zGEa6fnkRz+B/kbpXvNOlHQ=
7	5C5E87A9-4A01-41F0-88B8-E12AD69A8A18	28207E7B-F177-4CA1-8707-D7A4729E699E	14654977-03FB-452D-AFC2-6D9F0D9ACE36	HLZT03+zuFW8oDc8EN9U/xpBo9oOzu7HJrCdffbPk7Y=
8	A4B070EE-D140-4C06-BB31-E2124FE9D019	14654977-03FB-452D-AFC2-6D9F0D9ACE36	28207E7B-F177-4CA1-8707-D7A4729E699E	jpawJmhT2bHW/dsHNMCGk3yivwS8JZzCs9txK3RJfj4=

Figure 5-1: OriginalPayload field content

5-4. System Results and Performance Analysis

After the successful completion of the functional tests, the performance of the system was measured to ensure that security does not negatively affect the user experience.

5-4.1. Encryption Latency and Speed

The time required for the hybrid encryption process was measured. Hybrid encryption is used because RSA is computationally expensive for large data, while AES is significantly faster.

- **Message Processing Time:** The average time taken to encrypt a message on the client-side, transmit it through the server, and decrypt it on the receiver's side was a fast process.
- **Key Generation:** The generation of a **2048-bit RSA** key pair during the registration process was observed to be quick, depending on the client's hardware. Since this process happens only once during registration, it does not affect daily usage.

5-4.2. Data Integrity and Accuracy

It was verified that the decryption process reproduces the exact original message without any character loss or formatting issues. The use of **Base64 encoding** for keys and ciphertext ensured that the data remains right while being transmitted over the **JSON-based** API.

5-5. Comparative Analysis with Previous Studies

To demonstrate the significance of the developed system, a comparative analysis is conducted between the proposed secure chat architecture and the standard models commonly found in existing literature and traditional applications. This comparison focuses on architectural design, cryptographic implementation, and data privacy.

This table shows the comparison between previous studies and the proposed solution

Table 5-1: Studies Comparison

	Standard Messaging Systems	**Proposed Secure Chat System**
System Architecture	Built on a Two-Tier (Client-Server) model	Constructed using a Multi-Layered (N-Tier) architecture for maximum isolation
Encryption Methodology	Relies primarily on Transport Layer Security (SSL/TLS)	Implements a Hybrid Encryption model (RSA-2048 + AES-256)
Key Management	Keys are generated and stored on the central server	Keys are Generated on the Client-side, private keys never leave the user's device
Database Security	Data is often stored in plain text or protected by basic DB permissions	Implements Encryption at Rest using a unique **DbMasterKey**
Real-time Communication	Often uses standard HTTP polling or basic WebSockets	Utilizes **SignalR** with encrypted payloads for secure, real-time duplex communication

The comparative results highlight several areas where the proposed system offers best security and privacy over traditional models.

1. **Architectural Resilience:**

In standard two-tier models, a vulnerability in the web server often leads directly to a database breach. In this research, the N-Tier architecture was implemented to prevent such direct access. By separating the Business Logic Layer (BLL) from the Data Access Layer (DAL), an additional security is established. It was observed during testing that unauthorized requests are intercepted at the logic layer before reaching the sensitive data structures.

2. **Cryptographic power:**

A major mistake in many secure applications is the reliance on the server for key management. In the proposed system, the **Web Crypto API** is utilized to ensure that the user remains the owner of the private key. This approach ensures that even if the server infrastructure is compromised, the intercepted messages cannot be decrypted. The use of **RSA-2048** for secure key exchange and **AES-256** for data encryption provides a robust defense against both current and emerging computational threats.

3. **Advanced Storage Protection:**

While traditional systems may protect data during transit, the "at rest" state is often ignored. In the developed system, the **DbMasterKey** approach was introduced. Every message payload is encrypted using a server-side master key before being written to the **SQL Server**. This mechanism provides a perfect layer of defense against "Inside Attacks" where a database administrator or an intruder with direct file access attempts to read the communication logs.

4. **Real-time Integrity:**

By integrating **SignalR** with encrypted payloads, the system ensures that the responsiveness of the application is not sacrificed for security. It was verified that the overhead introduced by the multiple encryption layers is minimal, allowing for a seamless user experience while maintaining a high security status.

5-6. Conclusions Derived from Results

Based on the extensive testing and the comparative analysis performed in the previous sections, several critical conclusions were drawn regarding the effectiveness of the proposed security framework.

5-6.1. Impact of N-Tier Architecture on Security

The implementation of a multi-layered architecture was proven to be a good defense mechanism. It was observed that by isolating the Data Access Layer (DAL) from the Presentation Layer (Web API), an additional barrier was created against direct database exploitation. Even in a simulated scenario where the API layer was partially compromised, the sensitive data within the SQL Server remained protected due to the separation of concerns and the internal encryption logic handled by the Business Logic Layer (BLL).

5-6.2. Efficiency of Client-Side Key Management

One of the most significant findings was the reliability of the **Web Crypto API** for managing user private keys. By generating and storing RSA keys locally within the browser's localStorage, the "Zero-Knowledge" principle was successfully achieved. It was concluded that this approach effectively eliminates the risk of a "Global Data Breach" because a compromise of the central server would not provide an attacker with the keys required to decrypt individual user conversations.

5-6.3. Balancing Security and User Experience (UX)

A common concern in encrypted systems is the potential for performance degradation. However, the results from the performance testing phase indicated that the use of Hybrid Encryption (combining the speed of AES with the security of RSA) maintained a high level of responsiveness. It was concluded that the overhead introduced by these security layers is negligible for real-time applications, provided that the encryption processes are optimized and handled asynchronously on the client-side.

5-6.4. Resilience against Modern Cyber Threats

The combination of Salted Hashing (BCrypt), Pepper, and End-to-End Encryption provided a robust defense against common attacks such as Rainbow Table attacks and Man-in-the-Middle interceptions. The testing phase confirmed that the system is resilient against these threats,

ensuring that user identities and their private communications remain confidential even under active monitoring.

5-7. Chapter Summary

In this chapter, the developed secure chat system was rigorously evaluated through various testing methodologies. The environment was carefully prepared to simulate real-world conditions, and the system's performance was measured across different scenarios. The results confirmed that the N-Tier architecture and Hybrid Encryption mechanisms operate harmoniously to provide a secure and efficient communication platform.

Furthermore, the comparative analysis demonstrated that the proposed system offers superior privacy and data integrity features compared to traditional models and previous studies. The chapter concluded the research objectives, proving that a high degree of security can be achieved without compromising system performance or the user experience.

Conclusion

Project Summary

The primary objective of this research was the development of a highly secure, real-time communication platform capable of protecting user privacy against modern cyber threats. This was achieved through the implementation of a Multi-Layered (N-Tier) Architecture and an End-to-End Encryption (E2EE) framework. Throughout the project, the integration of SignalR for duplex communication and a Hybrid Cryptographic Model (RSA and AES) was successfully demonstrated. The system ensures that messages remain encrypted from the moment they leave the sender's device until they are decrypted by the intended recipient.

Creative Parts and Benefits

In this work, several creative approaches were introduced to enhance the standard security models:

- **Decentralized Key Management:** Unlike traditional systems where the server manages encryption keys, this project successfully shifted the responsibility to the client-side. The use of the **Web Crypto API** for local key generation ensures that private keys are never transmitted over the network.
- **Architectural Isolation:** The application of the N-Tier design pattern provided a clear separation between data access and business logic, reducing the attack surface compared to two-tier architectures.

Evaluation of Results: Strengths and Challenges

Strengths

The results obtained from the testing phase indicate that the system provides an exceptionally high level of confidentiality. The Hybrid Encryption model proved to be efficient, as it combined the robust security of asymmetric encryption with the high performance of symmetric encryption. It was verified that the system remains responsive, with minimal latency during real-time message exchange, proving that high security does not necessarily compromise the user experience.

Challenges and Weaknesses

Despite the success of the system, a challenge was identified related to Key Persistence. Since the private keys are stored within the browser's local storage to ensure privacy, the loss of this data (e.g., through clearing browser cache or changing devices) results in the permanent inability to

decrypt historical messages. While essential for absolute privacy, presents a challenge for long-term data recovery.

Personal Perspective and Viewpoints

From the researcher's perspective, the transition toward **Client-side Encryption** is no longer optional but a necessity in the era of cloud computing. It is believed that user privacy should be a fundamental architectural requirement rather than a secondary feature. The development of this project has demonstrated that building such systems requires a deep understanding of the synergy between frontend security and backend structural integrity.

Recommendations and Future Horizons

To further advance the security and usability of the platform, the following recommendations are proposed for future projects:

- **Biometric Key Recovery:** Future studies could investigate the use of biometric authentication to encrypt and back up private keys to a secure cloud vault, allowing for key recovery without compromising the E2EE principle.
- **Multi-Device Synchronization:** Research should be conducted on secure protocols for synchronizing private keys across multiple user devices (e.g., mobile and desktop) using secure "Handshake" mechanisms.
- **File and Media Encryption:** The current framework can be expanded to include the secure, chunked encryption of large files and multimedia content using the same hybrid logic.

In conclusion, this research has successfully provided a robust model for secure communication. By combining modern architectural patterns with advanced cryptographic standards, a system was produced to not only meets but exceeds the basic requirements for data confidentiality and integrity in the digital world.

References

[1]. Aman, F., Siddesh, D. N., Priyanka, H. M., Sinchana, H. K., & Patil, V. (2026). AI-based cyber security intrusion detection system. JAAFR Journal of Advance and Future Research, *4*(1), 253–258

[2]. Kumar, P., Singh, P., Pankaj, Singh, N., & Kumar, V. (2024). Encrypted communication with intelligent threat detection: A secure chat framework. International Journal of Research - GRANTHAALAYAH, 12(7), 211–220

[3]. Song, J., Qin, G., Liang, Y., Yan, J., & Sun, M. (2024). SIDiLDNG: A similarity-based intrusion detection system using improved Levenshtein Distance and N-gram for CAN. Computers & Security, 142, 103847

[4]. Ouarda, L., Malika, B., & Brahim, B. (2023). Towards a better similarity algorithm for host-based intrusion detection system. Journal of Intelligent Systems, 32, Article 20220259

[5]. Priya, B., Kumar, B. S. A., & Akhil, P. (2025). A secure messaging platform with advanced protection against MITM attacks and intrusion detection/prevention using machine learning. International Advanced Research Journal in Science, Engineering and Technology (IARJSET)

[6]. Fastowski, A., Prenkaj, B., Li, Y., & Kasneci, G. (2025). Injecting falsehoods: Adversarial man-in-the-middle attacks undermining factual recall in LLMs. arXiv preprint arXiv:2511.05919

الجامعة السورية الخاصة
كلية هندسة الذكاء الاصطناعي
قسم هندسة أمن النظم والشبكات الحاسوبية

Secure Chat-Based Environment Using AI-Based IDS

إعداد:

وليم راتب البني

إشراف:

د. وسيم الجنيدي

العام الدراسي

2026-2025